Strategies for Span Labeling with Large Language Models

Danil Semin semin@ufal.mff.cuni.cz
Ondřej Dušek odusek@ufal.mff.cuni.cz
Zdeněk Kasner kasner@ufal.mff.cuni.cz

Charles University, Faculty of Mathematics and Physics
Institute of Formal and Applied Linguistics
Prague, Czechia

Abstract

Large language models (LLMs) are increasingly used for text analysis tasks, such as named entity recognition or error detection. Unlike encoder-based models, however, generative architectures lack an explicit mechanism to refer to specific parts of their input. This leads to a variety of ad-hoc prompting strategies for input text span labeling, often with inconsistent results. In this paper, we categorize these strategies into three families: *tagging* the whole input text, *indexing* numerical positions of spans, and *matching* generated span content. To address the limitations of content matching, we introduce LogitMatch, a new constrained decoding method that forces the model's output to align with valid input spans. On four diverse tasks, we find that tagging-based methods are the most robust, but *matching* methods are competitive and more token-efficient. Our LogitMatch approach improves upon *matching* methods by retaining their performance while eliminating span matching issues.[1]

1 Introduction

Generative large language models (LLMs) are mostly based on the Transformer decoder architecture (Vaswani et al., 2017; Radford et al., 2019), which makes them well-equipped for open-ended generative tasks. However, LLMs are increasingly used for text analysis tasks as well: from finding factual inconsistencies or grammatical errors to extracting structured information about events and entities. In these tasks, an issue arises: **how can LLMs explicitly refer to parts of their input text?**

While constraining the model output structure is a commonly supported feature in LLM inference frameworks (Liu et al., 2024; Willard and Louf, 2023; guidance-ai, 2025), there is no straightforward way to constrain the model output to contain only parts of the given input text. Unlike encoder-based models (Devlin et al., 2019; Warner et al., 2025) that can attach a label to each input token, generative LLMs, by contrast, have no explicit mechanism to refer to their input. This issue becomes apparent in span labeling tasks, where we need to identify and categorize parts of the given text (see Figure 1 for an example). In these cases, we find ourselves needing to *refer to spans in the given input text by generating another text*, which offers no guarantees on alignment between the two texts.

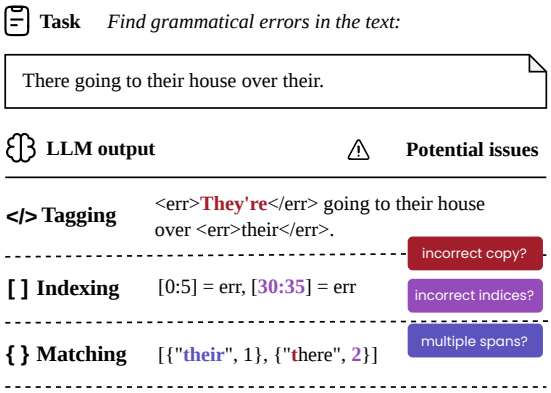


Figure 1: Main span labeling strategies: tagging, matching, and indexing. The issues include inability to copy the input text verbatim, wrong numerical indices, and ambiguities stemming from multiple identical spans.

Why use LLMs for span labeling? First, LLMs can achieve similar performance on span labeling tasks to finetuned encoder models, without costly task-specific training (Wang et al., 2025a; Kocmi et al., 2024). Thanks to their in-context learning abilities, LLMs can also deal with low-resource tasks for which custom-finetuned encoder models are not readily available, such as evaluating factual accuracy of sports reports (Thomson et al., 2023), identifying rhetorical structures (Mulcaire and Madnani, 2025), or detecting fallacies and propaganda techniques (Hasanain et al., 2024; Ram-

[1] Our code is available at: https://github.com/semindan/span_labeling.

Proceedings of the 19th International Natural Language Generation Conference: Long Papers, pages 229–245
October 17-21, 2026 ©2026 Association for Computational Linguistics

poni et al., 2025).

In this paper, we explore how to robustly apply LLMs to span labeling tasks. Our goal is not to establish whether generative LLMs outperform finetuned encoder-based taggers, but to compare prompting strategies for practitioners who already rely on off-the-shelf generative LLMs for span labeling. We first show that there is currently no consensus on how to formulate span labeling tasks for LLMs. This leads to a wide range of ad hoc strategies, often found to deliver unsatisfactory performance on downstream tasks. In Section 3, we cluster these strategies into three groups: tagging a copy of the whole input text (*tagging*), referring to the numerical indices of the input tokens (*indexing*), and matching the generated span content (*matching*).

In Section 4, we introduce our novel constrained decoding extension for the *matching* strategies. Our method, dubbed LOGITMATCH, provides guarantees on the output format by modifying raw model logits to only allow generating valid spans from the input text. By only affecting the decoding process, our method is applicable to any locally-deployed LLM without costly model finetuning or architecture modifications.

In Section 5, we evaluate a representative set of span labeling methods across four diverse tasks: named entity recognition, grammatical error correction, machine translation error detection, and a synthetic pattern lookup task. In Section 6, we discuss our results, showing that while *tagging* strategies offer a robust baseline, LOGITMATCH successfully addresses the limitations of standard *matching* strategies and offers a highly competitive approach. We also analyze the trade-offs of each method and provide recommendations regarding their applications to specific tasks.

2 Task Definition

In this paper, our goal is to (1) **identify spans**, i.e., contiguous sequences in an input text satisfying given criteria and (2) **classify each span** into one of predefined categories. Both of these subtasks need to be accomplished by generating textual output, in order to apply LLMs without architecture modifications.

Formally, we define the problem as follows: Given an input text $X = [x_1, x_2, \ldots, x_n]$ consisting of n characters, the objective is to identify and label a set of spans $S = \{(b_i, e_i, c_i)\}_{i=1}^{m}$ in X satisfying given criteria. Each span is characterized by three components: a begin index b_i, an end index e_i (exclusive), and a category c_i from the set of span categories C defined for the task. The begin and end indices satisfy $1 \leq b_i \leq e_i \leq n + 1$. In this work, we consider the spans to be non-overlapping. We are using a model $\mathcal{M}$ that generates an output string $Y = [y_1, y_2, \ldots, y_{|Y|}]$, out of which S needs to be extracted.

3 Existing Strategies for Span Labeling with Generative Models

We identified three groups of strategies in the literature that are used to formulate span labeling for generative models (see Table 1 for an overview). We categorize these strategies depending on how the spans are marked: *tagging* (Section 3.1), *indexing* (Section 3.2), and *matching* (Section 3.3).

3.1 *Tagging*: Marking Spans in the Input Text

Tagging strategies explicitly generate an output for each input token, requiring $O(n)$ output tokens per example. In the more common variant, the model generates a full copy of the input text and surrounds the spans of interest with special tags. Alternatively, the model can generate a sequence of tags, one tag per input token, and include *null* tags for unmarked tokens.

Custom Tags One of the first applications of LLMs for span labeling is the work of Wang et al. (2025b), who apply an LLM to identifying named entities in a text. They prompt the model to surround the entities with special markers `@@` and `##` to denote tag boundaries. Yan et al. (2024) extend their approach by instructing the model to append the entity category after the closing tag (such as `##LOC`), adding the possibility to classify the spans.

XML-like Tags To detect translation errors, Treviso et al. (2024) instruct an LLM to surround errors with XML-like tags such as `<error1 severity="major">text</error1>`. A similar approach is adopted by Mulcaire and Madnani (2025) to detect shell language (filler words and phrases) in English language exams.

BIO Tags The BIO scheme (Ramshaw and Marcus, 1995) classifies each token as a beginning of a span (`B`), inside a span (`I`), or outside a span (`O`). Obeidat et al. (2025) use the BIO scheme for biomedical NER by making the LLM generate a pair `word:tag` for all words in the input text. The

Target output: Turing [PER] was born in London [LOC].

Str.	Method	Output	Used in
tagging	Custom tags	`@@Turing## was born in @@London##.`	Wang et al. (2025b)
		`@@Turing##PER was born in @@London##LOC.`	Yan et al. (2024)
	XML tags	`<entity type="PER">Turing</entity> was born in <entity type="LOC">London</entity>.`	Treviso et al. (2024), Mulcaire and Madnani (2025)
	BIO tags	`(Turing, B-PER) (was, O) (born, O) (in, O) (London, B-LOC)`	Obeidat et al. (2025), Ramponi et al. (2025)
		`B-PER O O O B-LOC`	Wang et al. (2025b)
indexing	Char-level	`[1:7] = PER` `[20:26] = LOC`	Hasanain et al. (2024)

	Word+num	*Input: [1]Turing [2]was [3]born [4]in [5]London*
		`[1:1] = PER`
		`[5:5] = LOC`
			Ramponi et al. (2025)

matching	Bullet points	`- PER - Turing` `- LOC - London`	Kocmi and Federmann (2023)
	JSON output	`[{"type": "PER", "text": "Turing"}, {"type": "LOC", "text": "London"}]`	Kasner and Dušek (2024), Kasner et al. (2025)
		`[{"type": "PER", "text": "Turing", "index": 0}, {"type": "LOC", "text": "London", "index": 0}]`	Klesnilová (2025), Hasanain et al. (2024)

Table 1: Overview of span labeling strategies with generative models used in related work. As an example, we present the methods applied on the NER task for the sentence "*Turing was born in London.*" with two span categories: `PER` (person) and `LOC` (location).

BIO tagging scheme was also initially tested by Ramponi et al. (2025) to detect errors in social media texts, but it was discarded due to unsatisfactory results. An alternative approach proposed by Wang et al. (2025b) also relies on BIO tags but only generates the sequence of tags without generating the words themselves. However, this underperforms their aforementioned custom tag approach.

3.2 *Indexing*: Referring to Character Indices

Indexing strategies output spans to be labeled, which requires $O(m)$ tokens per example (i.e., grows linearly with the number of annotated spans). The model is explicitly asked for the numerical begin and end indices (b_i, e_i) of each span.

Character-level For propaganda span detection, Hasanain et al. (2024) ask for the start and end character indices of the span. They report that the method led to inaccuracies since the model did not have explicit access to character indices. The authors eventually resorted to string matching of the span content (cf. Section 3.3).

Word-level with numbering For the task of detecting argumentation fallacies, Ramponi et al. (2025) provide an LLM with explicit access to word indices by appending an index to each word in the input text. The output they require is in the format `[start_word:end_word = category]`. This approach was found to perform better than using BIO tags (cf. Section 3.1).

3.3 *Matching*: String Matching of the Span Content

Matching strategies generate the textual content of each span. As such, they generate $O(m \cdot \bar{l})$ tokens per example, where $\bar{l}$ is the average length of span content.[2] Unlike in *indexing* strategies, the model generates the textual content of the spans. The begin and end indices (b_i, e_i) are determined post-hoc by string matching.

Unstructured List A basic version of this approach is adopted by Kocmi and Federmann (2023), who detect translation errors by asking the model to generate a bulleted list with items `{error_category} - {span_text}`.

JSON Output Kasner and Dušek (2024) use a JSON-like format for evaluating output accuracy in data-to-text generation. The approach is also applied by Kasner et al. (2025) to identify errors in machine translation and detect propaganda

[2] Since the span content can cover the entire input text in the extreme, the upper bound is $O(n)$. However, the spans tend to be much shorter in practice.

techniques. Similarly, Hasanain et al. (2024) and Klesnilová (2025) use the JSON format to detect propaganda techniques. To disambiguate spans with the same content, Klesnilová proposes adding a JSON field `occurrence_index` with the sequential number of the span.

3.4 Issues with Existing Methods

As shown in Figure 1, all of the strategies are problematic. Some of the drawbacks are inherent, while others are related to specific limitations of LLMs.

LLMs cannot reliably *copy*. Both the *tagging* and *matching* strategies rely on the model's ability to copy its input. For *tagging*, the model must reproduce the entire input text exactly to preserve alignment. For *matching*, the generated span must match the input substring character-for-character. In practice, however, LLMs tend to "fix" the text by correcting typos, changing capitalization, or correcting factually incorrect sentences (Liu et al., 2024; Li et al., 2024; Jarolím et al., 2025). It is possible to partially mitigate this issue with fuzzy matching during post-processing, but that may introduce a new source of errors and does not solve the underlying issue.

LLMs cannot reliably *count*. Although positional embeddings allow LLMs to operate with relative positions of input tokens, the embeddings do not provide the model with a straightforward access to explicit token indices. The knowledge of number of characters in each token (or number of tokens per word) is learned indirectly (Edman et al., 2024; Fu et al., 2024). Therefore, when *indexing* strategies attempt to bypass text generation by predicting numerical positions directly, the indices are only "informed guesses". One can mitigate this by inserting indices into the text explicitly (Ramponi et al., 2025). However, that disrupts the natural flow of the text and potentially degrades model performance.

Span content may be ambiguous. For *matching* strategies, the lack of grounding in the input text can also introduce ambiguities. If the model correctly identifies a span (e.g., "the"), but that exact span appears many times across the document, string matching alone cannot determine which specific occurrence the model intended to label. Disambiguating by asking the model to generate an occurrence index (where the model can suggest that it refers to the *second* instance of word "the"; Klesnilová, 2025) once again places the burden on the model's counting abilities.

4 LogitMatch: Making *Matching* Methods More Robust

We introduce a new method, LogitMatch, that addresses the first weakness listed in Section 3.4 for *matching* strategies, i.e., the fact that the model may not perfectly reproduce the input span. Our method ensures that each decoded span is a valid input span while making only minimal interventions to the model's decoding process.

We implement LogitMatch for decoding a list of spans in a JSON format (cf. Section 3.3). We ask the model to provide a list of JSON objects, where each object contains the field `text` with the span content.[3] The main idea of LogitMatch is to **limit the set of permissible tokens while the `text`[4] field is generated** to ensure that the decoded tokens are a continuous span from the input text.

Our method is orthogonal to the structured decoding approaches (Willard and Louf, 2023), which constrain the overall syntactical structure of the response, but cannot ensure that the *content* of a specific field is a valid input substring. In fact, our approach is fully compatible with structured outputs; we combine the two methods in our experiments (cf. Section 5.1).

4.1 Algorithm

Our algorithm (see Appendix C.2 for pseudocode) uses three modes: `DEFAULT`, `SELECT`, and `COPY`.

Default Mode Our algorithm starts in `DEFAULT` mode, where it uses standard autoregressive decoding, sampling the next token from the model distribution: $y_t \sim P_{\mathcal{M}}(y_t \mid y_{<t}, X)$.

Select Mode The `SELECT` mode is activated once the model starts decoding the value of the `text` field containing the span content. In `SELECT` mode, we limit the model's vocabulary to tokens present in the tokenized input $[x_1, x_2, \ldots, x_n]$ (here x_i denotes the i-th input *token*, not a character as in Section 2) and let the model generate a *single token* x_i starting a span.

[3] The objects also contain other fields; we decode them without constraints. We only limit the `label` field we use for span categorization to contain valid labels.

[4] The field naming we selected is arbitrary and does not affect the generality of our method.

Copy Mode After the token x_i is generated, we switch to the COPY mode, where the model can generate the following input token x_{i+k} (where initially $k = 1$) or a closing quote ("). If the model generates the token x_{i+k}, we increment k and continue in the COPY mode. In the latter case, we exit the COPY mode and continue in the DEFAULT mode.

4.2 Tokenization Issues

Tokenization introduces additional aspects that need to be handled with care in LOGITMATCH to avoid disallowing valid tokens.

Different input and output tokenization. We need to account for the fact that tokenization of the output text does *not* necessarily correspond to the input text tokenization. Consider, for example, the input text "Hello.". On the input, the text may be tokenized as ["Hello", "."]. However, the model might first decode the token "Hel", which guides it towards the tokens "lo" and ".". Therefore, in the COPY mode, we need to allow not only the following input token, but *all tokens that are valid prefixes of the subsequent text*.

Handling the field boundaries. The quotes delimiting a JSON value may be part of a token containing other content-related characters. When entering the SELECT mode (i.e., in the step where the expression "text"\s:\s" would be decoded), we need to consider whether the token containing the opening quote also has a suffix (e.g. "London) and handle the suffix (in this case London) in the context of the SELECT mode. Accordingly, in the COPY mode, we need to whitelist all the tokens that satisfy the following: they start with the valid prefix, i.e., continuation of the decoded text, followed by a quotation mark, and optionally followed by a suffix (e.g., a comma followed by a newline); all these tokens exit the COPY mode.

5 Experiments

5.1 Methods

Based on our survey in Section 3 (see also Table 1), we implement and evaluate what we consider a representative set of methods for each strategy:

- TAG: *tagging* with XML-like tags.
- INDEX: *indexing* with character-level indices.
- INDEX-ENRICHED: *indexing* with character-level indices, with the character indices inserted before each word on the input.

Task	# Ex.	# Lang.	# Cat.	Word tok.
NER	7,523	13	3	✓
GEC	504	1	3	✓
ESA-MT	867	9	2	✗
CPL	1,000	1	1	✗

Table 2: Statistics of the datasets used in our experiments. The table shows the number of examples, languages, and categories used for each task. The inputs for NER and GEC use NLTK-style word tokenization.

- MATCH: *matching* with JSON output.
- MATCH-OCC: *matching* with JSON output and occurrence index.

We also implement our novel method from Section 4 analogically to the *matching* methods:

- LOGITMATCH: *matching* with JSON output and constrained decoding.
- LOGITMATCH-OCC: *matching* with JSON output, constrained decoding, and occurrence index.

For JSON-based outputs, we additionally test a structured output variant (denoted with -S) in which we enforce the JSON schema using structured output.[5] In this variant, the model output is guaranteed to contain all the required fields and a valid category label.

We parse the outputs of *tagging* and *indexing* approaches using regular expressions; *matching* approaches use JSON parsing. See Appendix C for more implementation details.

5.2 Datasets

For our experiments, we select four span labeling tasks. First, we use two well-established high-resource NLP tasks: named entity recognition and grammatical error correction. We also adopt span-level detection of errors in machine translation as a low-resource span labeling task. Finally, we create a synthetic task called *conditional pattern lookup* to stress test robustness of all approaches. See Table 2 for an overview of our datasets.

Named Entity Recognition (NER) The goal in the NER task is to localize named entities and classify their type. We use the UniversalNER benchmark (Mayhew et al., 2024) that covers three coarse-grained entity types: Person (PER), Organization (ORG), and Location (LOC). The dataset contains 7,523 examples across thirteen languages.

[5] https://docs.vllm.ai/en/stable/features/structured_outputs/

Grammatical Error Correction (GEC) In the GEC task, the goal is to identify spans requiring edits in order to make the text grammatically correct. We adapt the English Write & Improve corpus from the MultiGEC dataset (Masciolini et al., 2025) that contains 504 examples with a list of edits between the original and corrected sentences from the ERRANT tool (Bryant et al., 2017). We use the categorization of grammatical errors into three high-level categories: replacement (`R`), missing (`M`), and unnecessary (`U`).[6]

Detecting Errors in Machine Translation (ESA-MT) In the ESA-MT task, the goal is to find error spans in translated texts given their source. We use data from the error span annotation (ESA) task at WMT24 (Kocmi et al., 2024) that contains translation of texts from the news, social and literary domains from English into nine languages. Specifically, we select examples containing errors from the test set of Kasner et al. (2025), resulting in 867 examples. Following the dataset's annotation scheme, we classify the errors as `MINOR` or `MAJOR`.

Conditional Pattern Lookup (CPL) In real-world tasks, multiple occurrences of the same span are relatively rare. To test how the approaches behave with multiple occurrences of a specific span, we devise a synthetic task called *conditional pattern lookup*. The goal of the task is to find occurrences of a given regular expression in the input sequence. We use constraints to make the lookup conditional: only some occurrences of the pattern should be identified based on the pattern it is preceded or followed by. For example, *Find all sequences matching* '`\w+ dry`' *that are not preceded by* '`house`'. Note that this can be easily solved by any regular expression matcher; we apply LLMs only for benchmarking purposes. As an input, we generate 1,000 random sequences of approximately 100 English words. We ensure that the pattern occurs at least once in the input sequence.

5.3 Models

We test all approaches with state-of-the-art open LLMs: Qwen3-8B (Yang et al., 2025), Mistral-Small-24B-Instruct (Mistral AI Team, 2025), Llama-3.3-70B-Instruct (Grattafiori et al., 2024), and Gemma-4-31B-IT (Google DeepMind, 2025). In Appendix B.6, we provide additional results with the proprietary GPT-5-mini model for methods that do not require logit access.

All models are run with 16-bit precision except for Llama, which is 4-bit quantized. We run each model with five different random seeds and average the results (see Appendix A for details and standard deviations). By default, we disable reasoning settings of the models. To test how reasoning may help, we performed additional experiments with Qwen3-8B with the reasoning effort set to `high`, which we denote as Qwen3-8B-Think; see Appendix B.5.

5.4 Evaluation Metric

Our evaluation metric extends the standard F1-score by counting matches proportionally based on their character-level overlap (Da San Martino et al., 2019; Kasner et al., 2025). Let C_p denote the set of character positions covered by predicted spans, C_g the set of character positions covered by gold spans, and $C_o = C_p \cap C_g$ the positions covered by both. Then:

$$\text{Precision} = \frac{|C_o|}{|C_p|}, \quad \text{Recall} = \frac{|C_o|}{|C_g|},$$
$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

We evaluate both *hard* and *soft* variants of this metric. In the hard variant, a position in C_o requires the gold and predicted spans covering it to share the same category label; the soft variant omits this requirement.

6 Results

We primarily base our observations on the overlap-adjusted *hard* F1-score in Table 3. We also provide the *soft* F1-score results in Table 10 in Appendix B. To give more insight into the failures of individual strategies, we also compute the following error rates (see Figure 2):

- **Response parsing errors**: The model output could not be parsed (e.g., invalid JSON).
- **Span matching errors**: The span content generated by the model could not be matched to any substring in the input text.
- **Label matching errors**: The category label generated by the model did not match any of the expected labels.

We summarize our main observations below.

[6] For more details on how we adapted GEC for span labeling, see Appendix C.3.

Method	NER 8B	NER 24B	NER 70B	NER 31B	GEC 8B	GEC 24B	GEC 70B	GEC 31B	ESA-MT 8B	ESA-MT 24B	ESA-MT 70B	ESA-MT 31B	CPL 8B	CPL 24B	CPL 70B	CPL 31B
TAG	73.3	74.1	**82.8**	88.9	**27.2**	**31.3**	**34.7**	**52.9**	10.2	11.0	**10.1**	**14.2**	40.2	53.1	79.5	93.3
INDEX	19.1	27.1	27.7	53.1	10.4	7.2	8.5	9.4	7.5	7.4	6.1	7.9	34.1	40.0	36.6	57.9
INDEX-ENRICHED	33.7	68.5	70.0	87.4	11.0	18.5	16.4	48.3	7.5	8.1	9.0	9.8	45.4	**70.1**	84.4	**98.6**
MATCH	**74.1**	**79.1**	81.7	87.4	12.1	15.5	20.1	38.4	11.4	12.2	9.2	12.4	42.8	43.8	48.5	51.1
MATCH-S	72.8	76.3	82.0	85.2	12.0	16.2	19.4	38.0	10.7	**12.6**	8.9	11.5	42.6	41.1	47.8	39.4
MATCH-OCC	73.0	78.3	81.6	88.2	15.4	18.6	20.8	38.6	**11.9**	12.4	9.4	13.0	**70.1**	69.1	**86.5**	94.7
MATCH-OCC-S	70.8	76.1	81.5	87.9	14.9	20.4	21.1	41.4	11.1	12.3	9.4	12.8	**70.1**	67.7	83.6	93.8
LOGITMATCH	73.3	76.3	79.8	89.2	17.4	23.0	19.3	41.7	11.2	11.9	9.0	13.1	40.1	42.9	47.4	51.1
LOGITMATCH-S	71.9	73.8	79.9	86.2	16.8	20.4	21.6	41.7	10.8	11.4	8.8	11.4	41.2	40.9	46.7	39.7
LOGITMATCH-OCC	72.4	75.2	79.1	**89.8**	21.3	24.4	20.5	45.1	11.8	11.6	8.8	13.5	68.6	68.9	86.3	94.7
LOGITMATCH-OCC-S	70.1	73.0	79.3	89.2	20.4	23.5	22.7	45.2	11.2	11.1	9.0	12.8	69.6	68.0	84.4	93.7

Table 3: Per-method **hard** F1 score in %. The best score for each model per dataset is bold.

Task: NER	**Model: Llama-3.3-70B**
Input	On Thursday , the 3rd of November , the mayor of Saint - Gaudens (in the Haute - Garonne region) decided to suspend the urban development agreement with the government formed in 2014 .
Match	`{"text": "Saint-Gaudens", "label": "LOC"}, {"text": "Haute-Garonne", "label": "LOC"}`
LogitMatch	`{"text": "Saint - Gaudens", "label": "LOC"}, {"text": "Haute - Garonne", "label": "LOC"}`
Task: ESA-MT (en-cs)	**Model: Gemma-4-31B-IT**
Source	... what type of plane you'll be on, it's more complicated to determine what that means...
Target (orig)	... jaký typ letadla budete letět, ale složitější je určit, co to znamená...
Target (enr.)	... 236::jaký 241::typ 245::letadla 253::budete 260::letět, 267::ale 271::složitější 282::je 285::určit...
Index	`[247:261] = MAJOR` ... *typ letadla budete letět,...*
Index-Enr.	`[253:266] = MAJOR` ... *letadla budete letět, ale...*

Table 4: Qualitative examples comparing different span labeling methods. In the NER task, the **Match** method fails to reproduce the input tokenization (spaces around hyphens), while **LogitMatch** correctly adheres to it. In the ESA-MT task, the **Index** method generates indices that are ignoring word boundaries, whereas **Index-enriched** helps the model locate the span correctly. Problematic outputs are highlighted in red.

LOGITMATCH improves matching performance in non-standard inputs. Our method improves results on the NER and GEC tasks where the input text is tokenized using traditional NLP conventions (e.g., spaces around punctuation). With vanilla *matching* methods, LLMs tend to normalize the text during generation, leading to mismatches (see Table 4 for an example of such behavior). At the same time, our method maintains the performance of the *matching* methods. In Appendix B.3, we further discuss the results of LogitMatch for other tasks.

Tagging is robust, but token-heavy. The TAG method outperforms other methods on GEC and shows competitive results for the other tasks.[7] However, *matching* methods are more token-efficient: as *tagging* methods must reproduce the full input text in every response, their cost is usually the highest, unless the spans collectively cover most of the input text (see Figure 3).

Indexing is not reliable without indices. Directly asking models to predict numerical indices (INDEX) leads to low performance on all tasks. As illustrated in Table 4, models frequently hallucinate indices that ignore word boundaries or fall entirely outside of the text. Adding explicit numbering to the input (INDEX-ENRICHED) can sometimes significantly improve the situation (e.g., by 24-52% points on NER). However, this method still underperforms other methods, especially with weaker models. A plausible explanation is that the inserted

[7] Note that this is affected by our post-processing choices. We align predicted spans to closest occurrences and thus smooth over imprecisions in source text copying. If source text exact match is required, the score drops by 9 points on average, but we believe this setting is too strict to be realistic for practical use. See Appendix B.2 for details.

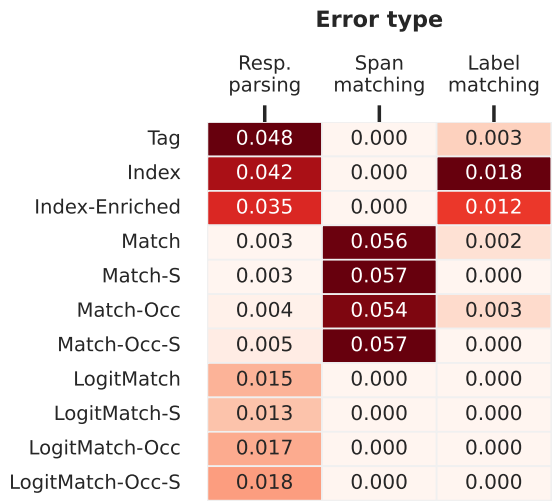


Figure 2: Error rates for each method (highlighted for each column separately), average for open LLMs across all datasets. We report three kinds of errors: error in parsing the model response, error in matching the span content, and error in matching the category label to one of the existing labels.

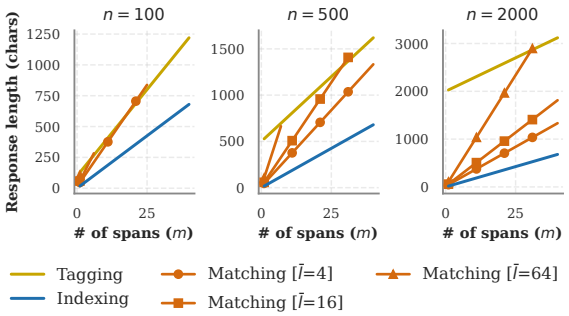


Figure 3: Estimated response length of individual methods based on the length of the input text (n), the number of spans (m) and the average length of span content ($\bar{l}$). Computation details are provided in Appendix B.4.

indices interfere with the ability of the model to process the meaning of the input text.

Using the `occurrence_index` field helps with matching multiple occurrences. In the CPL task, the performance of *matching* methods improves dramatically (by ~30-50% points) by using the `occurrence_index` for indexing the span content. This makes sense as *matching* methods otherwise have no way to uniquely refer to individual occurrences of the span on the input. However, for tasks where the spans are mostly unique and unambiguous, such as NER and ESA-MT, this feature does not provide a clear benefit.

Structured outputs do not uniformly improve performance. While structured outputs expectedly minimize parsing errors[8] (see Figure 2), restricting an output format naively may hurt model performance. Specifically, we find that the models *without* structured outputs sometimes produce a "spontaneous" chain-of-thought before generating the output, which can help the model to improve its answer (at the cost of producing more output tokens).

Explicit reasoning helps, but slows down inference. The issue with restricting the output does not arise with reasoning models that explicitly enclose their reasoning trace between `<think>` tags, as this reasoning trace is handled separately in LLM frameworks.[9] Our additional experiments (see Appendix B.5) show that reasoning may boost the model performance – for example, by explicitly indexing individual characters in the reasoning trace. However, this comes at the cost of a much higher number of output tokens during inference, which in our case sometimes lead to hitting the maximum output token limit before producing the answer.

Model size is not the only factor. Since our primary goal was to explore span labeling strategies rather than benchmark models on the tasks, comparisons of model performance should be interpreted with caution. Our results suggest that Qwen3-8B is competitive with the much larger Llama-3.3-70B, and that Gemma-4-31B-IT achieves the highest scores overall on most tasks despite being a 31B model. Factors such as model release date, quantization, and training data may all influence these results.

7 Conclusion

We identified three major strategies for span labeling with LLMs and tested them on four span labeling tasks using state-of-the-art LLMs. Overall, the straightforward strategy of *tagging* the spans with XML-like tags has the most consistent results. However, *matching* strategies are more token-efficient and competitive overall. The *indexing*

[8]The error rate is non-zero since the models may still fail to return a valid JSON, e.g., if they get stuck in an infinite loop and reach the maximum number of output tokens.

[9]Note that LogitMatch constrains only the final answer portion of the output (after the `</think>` closing tag); the reasoning trace itself is generated without any vocabulary constraints.

strategies can be competitive only if the input is enriched with indices. We proposed a new method, LOGITMATCH, that solves the alignment issue of *matching* methods. Future work might explore how to combine the advantages of individual methods and how prompting with different examples and instructions influences the efficiency of each method.

Limitations

We focused only on span labeling approaches that involve parsing output text, as we find these the most generally applicable. Therefore, we did not explore alternative approaches that are more resource intensive, such as LLM2Vec (BehnamGhader et al., 2024) that transforms decoder-only LLMs into encoders with additional finetuning, or the approach of Dukić and Šnajder (2024) that allows the LLM to join span labeling with input processing by means of architecture modifications. Relatedly, our comparison is scoped to generative LLMs used off-the-shelf via prompting; we do not compare against finetuned encoder-based taggers, which require supervised training per task and language.

We use a single fixed prompt template per method (Appendix C.1); systematically varying prompt wording to test prompt-robustness is a substantial undertaking that we leave to future work. Our central claims are comparative – which output format is most robust for a fixed prompt scaffold – and since all methods share the same underlying prompt template structure, we do not expect a uniform change in prompt wording to overturn the relative rankings we report.

Our proposed method does not address the disambiguation of multiple identical spans. As we have shown, our method can be combined with `occurrence_index` to achieve that effect. However, this approach is not perfect. Future work might thus explore other approaches, for example, disambiguating the span using attention values or by re-generating adjacent text.

In principle, a variant of LOGITMATCH is also applicable to *tagging* methods, where it may constrain the decoded text to correspond to the input text. However, our preliminary experiments ran into implementation issues with tokenization handling, so we abandoned this approach. Future attempts should carefully handle tokens at the boundaries between the outer text, the tag itself, and the inner text to avoid performance degradation.

Finally, we note that NER and GEC are well-established tasks for which large amounts of BIO-tagged and XML-annotated training data are publicly available. Pre-training exposure to these annotation formats may advantage certain methods on those tasks. We selected ESA-MT and CPL as lower-resource tasks with less standard annotation formats to partially mitigate this effect, and we observe that the relative method rankings do indeed differ across tasks.

Ethical Considerations

To the best of our knowledge, our work does not pose any immediate ethical risks. Our proposed method is designed to improve the reliability of text analysis tools, which we believe is a helpful contribution to the field. For our experiments, we relied solely on existing, publicly available datasets. We did not collect any new data or employ human annotators for this project. We used AI assistants to help with writing experimental code and to improve the clarity of the text. All generated content was manually reviewed and verified by the authors.

Acknowledgments

This work was funded by the European Union (ERC, NG-NLG, 101039303), the Charles University Research Centre program No. 24/SSH/009, and the National Recovery Plan funded project MPO 60273/24/21300/21000 CEDMO 2.0 NPO. It used resources of the LINDAT/CLARIAH-CZ Research Infrastructure (Czech Ministry of Education, Youth, and Sports project No. LM2023062).

References

Parishad BehnamGhader, Vaibhav Adlakha, Marius Mosbach, Dzmitry Bahdanau, Nicolas Chapados, and Siva Reddy. 2024. Llm2vec: Large language models are secretly powerful text encoders. In *First Conference on Language Modeling*.

Christopher Bryant, Mariano Felice, and Ted Briscoe. 2017. Automatic Annotation and Evaluation of Error Types for Grammatical Error Correction. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics, ACL 2017 4, Volume 1: Long Papers*, pages 793–805, Vancouver, Canada.

Giovanni Da San Martino, Seunghak Yu, Alberto Barrón-Cedeño, Rostislav Petrov, and Preslav Nakov. 2019. Fine-Grained Analysis of Propaganda in News Article. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*

and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pages 5636–5646. Association for Computational Linguistics.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, MN, USA.

David Dukić and Jan Šnajder. 2024. Looking right is sometimes right: Investigating the capabilities of decoder-only llms for sequence labeling. In *Findings of the Association for Computational Linguistics ACL 2024*, pages 14168–14181.

Lukas Edman, Helmut Schmid, and Alexander Fraser. 2024. CUTE: Measuring LLMs' Understanding of Their Tokens. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL*, pages 3017–3026, USA.

Tairan Fu, Raquel Ferrando, Javier Conde, Carlos Arriaga, and Pedro Reviriego. 2024. Why Do Large Language Models (LLMs) Struggle to Count Letters? *CoRR*, abs/2412.18626.

Google DeepMind. 2025. Gemma 4. Apache 2.0 license.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, and 1 others. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

guidance-ai. 2025. Guidance: A language for constraining large language models. `https://github.com/guidance-ai/guidance`. MIT License.

Maram Hasanain, Fatema Ahmad, and Firoj Alam. 2024. Large Language Models for Propaganda Span Annotation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 14522–14532, Miami, Florida, USA.

Antonín Jarolím, Martin Fajčík, and Lucia Makaiová. 2025. Can LLMs extract human-like fine-grained evidence for evidence-based fact-checking? *arXiv preprint arXiv:2511.21401*.

Zdeněk Kasner and Ondřej Dušek. 2024. Beyond Traditional Benchmarks: Analyzing Behaviors of Open LLMs on Data-to-text Generation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024*, pages 12045–12072, Bangkok, Thailand.

Zdeněk Kasner, Vilém Zouhar, Patrícia Schmidtová, Ivan Kartáč, Kristýna Onderková, Ondřej Plátek, Dimitra Gkatzia, Saad Mahamood, Ondřej Dušek, and Simone Balloccu. 2025. Llms as span annotators: A comparative study of llms and humans. *Preprint*, arXiv:2504.08697.

Kristýna Klesnilová. 2025. Multilingual Detection of Persuasion Techniques in Text using Large Language Models.

Tom Kocmi and Christian Federmann. 2023. GEMBA-MQM: Detecting Translation Quality Error Spans with GPT-4. In *Proceedings of the Eighth Conference on Machine Translation, WMT 2023*, pages 768–775, Singapore.

Tom Kocmi, Vilém Zouhar, Eleftherios Avramidis, Roman Grundkiewicz, Marzena Karpinska, Maja Popovic, Mrinmaya Sachan, and Mariya Shmatova. 2024. Error Span Annotation: A Balanced Approach for Human Evaluation of Machine Translation. In *Proceedings of the Ninth Conference on Machine Translation, WMT 2024, Miami, FL*, pages 1440–1453, USA.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz*, pages 611–626, Germany.

Yongqi Li, Mayi Xu, Xin Miao, Shen Zhou, and Tieyun Qian. 2024. Prompting Large Language Models for Counterfactual Generation: An Empirical Study. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation, LREC/COLING 2024, 20-25 May, 2024*, pages 13201–13221, Torino, Italy.

Michael Xieyang Liu, Frederick Liu, Alexander J. Fiannaca, Terry Koo, Lucas Dixon, Michael Terry, and Carrie J. Cai. 2024. "We Need Structured Output": Towards User-centered Constraints on Large Language Model Output. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems, CHI EA 2024*, pages 10:1–10:9, Honolulu, HI, USA.

Arianna Masciolini, Andrew Caines, Orphée De Clercq, Joni Kruijsbergen, Murathan Kurfal, Ricardo Muñoz Sánchez, Elena Volodina, Robert Östling, Kais Allkivi, Špela Arhar Holdt, Ilze Auzina, Roberts Dargis, Elena Drakonaki, Jennifer-Carmen Frey, Isidora Glišić, Pinelopi Kikilintza, Lionel Nicolas, Mariana Romanyshyn, Alexandr Rosen, and 11 others. 2025. Towards Better Language Representation in Natural Language Processing: A Multilingual Dataset for Text-level Grammatical Error Correction. *International Journal of Learner Corpus Research*, 11:309–335.

Stephen Mayhew, Terra Blevins, Shuheng Liu, Marek Suppa, Hila Gonen, Joseph Marvin Imperial, Börje

Karlsson, Peiqin Lin, Nikola Ljubesic, Lester James V. Miranda, Barbara Plank, Arij Riabi, and Yuval Pinter. 2024. Universal NER: A Gold-standard Multilingual Named Entity Recognition Benchmark. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), NAACL 2024*, pages 4322–4337, Mexico City, Mexico.

Mistral AI Team. 2025. Mistral Small 3. https://mistral.ai/news/mistral-small-3. Accessed: 2026-01-02.

Phoebe Mulcaire and Nitin Madnani. 2025. Span Labeling with Large Language Models: Shell vs. Meat. In *Proceedings of the 20th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2025)*, pages 850–859, Vienna, Austria.

Motasem S. Obeidat, Md Sultan Al Nahian, and Ramakanth Kavuluru. 2025. Do LLMs Surpass Encoders for Biomedical NER? In *13th IEEE International Conference on Healthcare Informatics, ICHI 2025*, pages 352–358, Rende, Italy.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, and 1 others. 2019. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9.

Alan Ramponi, Agnese Daffara, and Sara Tonelli. 2025. Fine-grained Fallacy Detection with Human Label Variation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2025 - Volume 1: Long Papers, Albuquerque*, pages 762–784, New Mexico, USA.

Lance A. Ramshaw and Mitch Marcus. 1995. Text Chunking using Transformation-based Learning. In *Third Workshop on Very Large Corpora, VLC at ACL 1995*, Cambridge, Massachusetts, USA.

Craig Thomson, Ehud Reiter, and Barkavi Sundararajan. 2023. Evaluating factual accuracy in complex data-to-text. *Comput. Speech Lang.*, 80:101482.

Marcos V. Treviso, Nuno Miguel Guerreiro, Sweta Agrawal, Ricardo Rei, José Pombal, Tânia Vaz, Helena Wu, Beatriz Silva, Daan van Stigt, and André F. T. Martins. 2024. xTower: A Multilingual LLM for Explaining and Correcting Translation Errors. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 15222–15239, Miami, Florida, USA.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pages 5998–6008.

Shuhe Wang, Xiaofei Sun, Xiaoya Li, Rongbin Ouyang, Fei Wu, Tianwei Zhang, Jiwei Li, Guoyin Wang, and Chen Guo. 2025a. GPT-NER: Named Entity Recognition via Large Language Models. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 4257–4275, Albuquerque, New Mexico, USA.

Shuhe Wang, Xiaofei Sun, Xiaoya Li, Rongbin Ouyang, Fei Wu, Tianwei Zhang, Jiwei Li, Guoyin Wang, and Chen Guo. 2025b. GPT-NER: Named Entity Recognition via Large Language Models. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 4257–4275, Albuquerque, New Mexico, USA.

Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini, Alexis Gallagher, Raja Biswas, Faisal Ladhak, Tom Aarsen, Griffin Thomas Adams, Jeremy Howard, and Iacopo Poli. 2025. Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025*, pages 2526–2547, Vienna, Austria.

Brandon T. Willard and Rémi Louf. 2023. Efficient Guided Generation for Large Language Models. *CoRR*, abs/2307.09702.

Faren Yan, Peng Yu, and Xin Chen. 2024. LTNER: Large Language Model Tagging for Named Entity Recognition with Contextualized Entity Marking. In *Pattern Recognition - 27th International Conference, ICPR 2024*, volume 15319 of *Lecture Notes in Computer Science*, pages 399–411, Kolkata, India.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 40 others. 2025. Qwen3 Technical Report. *CoRR*, abs/2505.09388.

A Experimental Setup

A.1 Models

Table 5 presents the overview of our models and their decoding parameters. We run the open models using vLLM (Kwon et al., 2023) in 16-bit precision, except for Llama 3.3 70B that we quantize to 4-bit precision. For structured outputs, we use the vLLM structured output feature.[10] The decoding parameters are the default parameters in the Huggingface configuration file. We set only the

[10] https://docs.vllm.ai/en/latest/features/structured_outputs/

`max_tokens` parameter manually to 4,096 (and increased it to 16,384 for the reasoning models, see Appendix B.5).

A.2 Multiple runs

For each experiment, we ran each configuration with 5 different random seeds (42–46) and report the mean across seeds. Averaging over method-model configurations, the mean cross-seed standard deviation of hard F1 in percentage points is 0.46 on NER, 0.96 on GEC, 0.51 on ESA-MT, and 0.98 on CPL (see Table 6). These values indicate that the overall ranking patterns are stable across random seeds.

B Additional Results

B.1 Soft F1

In Table 10, we present the *soft* F1-score results of our models, complementing the *hard* F1-score results in Table 3. The *soft* F1 score does not penalize differences in category labels, only in span positions.

B.2 Tagging without Alignment

The TAG post-processing described in Appendix C.2 connects a predicted span to the closest exact occurrence of the identical text when the tag position does not align with the input exactly. To quantify how much this alignment step contributes to TAG's results in Table 3, we re-ran the TAG evaluation for the four open models with the alignment step removed, i.e., a predicted span is kept only if the text at its implied offset matches the tag content exactly, and dropped otherwise. Table 7 reports the hard F1 scores of this ablation alongside the aligned TAG, MATCH, and INDEX results from Table 3.

Removing alignment lowers TAG's hard F1 by about 9 points on average across the 16 task-model cells (mean aligned 48.6 vs. no-align 39.7), and without it, the gap between TAG and MATCH narrows considerably or reverses on several task-model combinations. TAG without alignment nonetheless still outperforms INDEX on most tasks, since the underlying reason for TAG's robustness – generating a full copy of the input rather than numerical positions – is unaffected by the alignment step itself.

The results demonstrate that our post-processing method has a non-trivial effect on the results reported for TAG in Table 3. However, without it, TAG fails in an all-or-nothing manner – a single misplaced character anywhere in the copied text throws off the running offset and can invalidate every subsequent span in the response. MATCH and LOGITMATCH, by contrast, fail independently span by span, so an error in one prediction does not propagate to the rest of the output. Since this brittleness is a straightforward implementation artifact of the *tagging* strategy (and one that a practitioner would most likely address), we consider the aligned variant more representative for our main results.

B.3 Decomposing Errors of Matching Methods

A legitimate concern that may arise from the results in Section 6 is that LOGITMATCH eliminates span matching errors (Figure 2) without a corresponding increase in hard F1 over MATCH. In Table 3, we decompose the soft and hard F1 scores of the *matching* methods together with their error rates, averaged over the four open models and all four tasks (non-thinking, unstructured configuration).

Table 8 shows that the soft-hard F1 gap is nearly identical for MATCH and LOGITMATCH (and likewise for their `-0cc` variants), even though LOGITMATCH reduces span matching errors to zero. This indicates that the spans MATCH fails to align to the input are largely spans on which the model's category prediction was already wrong, so removing span matching errors does not by itself close the soft-hard gap or lift hard F1. LOGITMATCH's slightly higher parse error rate stems from cases where the constrained decoding process does not terminate the JSON output within the token budget.

To conclude, this lack of correspondence between fewer span matching errors and higher hard F1 is arguably tied to the specific datasets in our evaluation suite rather than being a general property of LOGITMATCH. In domains with more frequent or more ambiguous repeated spans, span matching and label errors may be less correlated, in which case eliminating the former could yield a larger hard F1 improvement. Confirming this on a broader range of datasets is left to future work.

B.4 Token Efficiency Analysis

Figure 3 plots the theoretical response length of each method as a function of n, m, and $\bar{l}$. The curves are derived analytically from our specific output formats:

- ***tagging*:** $n + m \cdot C_{\text{tag}}$ — the response reproduces the full input text with XML-like tags around

Model	Model ID	temp.	top-p	top-k
Qwen3-8B	`Qwen/Qwen3-8B`	0.6	0.95	20
Mistral-Small-24B-Instruct	`mistralai/Mistral-Small-24B-Instruct-2501`	0.15	1.0	-
Llama-3.3-70B-Instruct	`meta-llama/Llama-3.3-70B-Instruct`	0.6	0.9	-
Gemma-4-31B-IT	`google/gemma-4-31b-it`	1.0	0.95	64
GPT-5-mini	`gpt-5-mini-2025-08-07`	1.0	1.0	-

Table 5: Model identifiers and their default decoding parameters that we used for our experiments (temperature, top-p, top-k).

Method	NER	GEC	ESA-MT	CPL
TAG	0.36	0.45	0.55	0.70
INDEX	0.28	0.49	0.78	1.65
INDEX-ENRICHED	0.77	0.64	0.59	1.51
MATCH	0.37	0.42	0.49	0.35
MATCH-S	0.73	0.41	0.45	2.00
MATCH-OCC	0.36	0.76	0.48	0.48
MATCH-OCC-S	0.32	0.71	0.51	0.66
LOGITMATCH	0.35	1.61	0.51	0.43
LOGITMATCH-S	0.65	1.52	0.40	1.80
LOGITMATCH-OCC	0.36	1.71	0.51	0.62
LOGITMATCH-OCC-S	0.37	1.64	0.35	0.57
Overall	0.45	0.94	0.51	0.98

Table 6: Mean cross-seed standard deviation of **hard** F1 in percentage points for each method and task, averaged across open models. The **Overall** row averages over all method-model configurations within a task.

Task	Method	8B	24B	70B	31B
NER	TAG (aligned)	73.3	74.1	82.8	88.9
	TAG (no-align)	59.4	47.1	75.2	81.0
	MATCH	74.1	79.1	81.7	87.4
	INDEX	19.1	27.1	27.7	53.1
GEC	TAG (aligned)	27.2	31.3	34.7	52.9
	TAG (no-align)	19.9	25.3	26.6	34.5
	MATCH	12.1	15.5	20.1	38.4
	INDEX	10.4	7.2	8.5	9.4
ESA-MT	TAG (aligned)	10.2	11.0	10.1	14.2
	TAG (no-align)	10.0	10.7	10.2	14.0
	MATCH	11.4	12.2	9.2	12.4
	INDEX	7.5	7.4	6.1	7.9
CPL	TAG (aligned)	40.2	53.1	79.5	93.3
	TAG (no-align)	8.2	52.0	74.5	86.0
	MATCH	42.8	43.8	48.5	51.1
	INDEX	34.1	40.0	36.6	57.9

Table 7: Hard F1 (%) of TAG with and without the post-hoc alignment step (Appendix C.2), compared to MATCH and INDEX, for the four open models (Qwen3-8B, Mistral-Small-24B, Llama-3.3-70B, Gemma-4-31B-it), average across datasets.

each span.

- ***indexing***: $m \cdot C_{\text{idx}}$ — only character-offset–label pairs are output.
- ***matching***: $C_{\text{base}} + m \cdot (C_{\text{span}} + \bar{l})$ — the JSON response contains a fixed structural wrapper plus per-span fields carrying the span text.

The constants are determined by our specific output templates (see Table 13), assuming a three-letter category name: $C_{\text{tag}} = 28$, $C_{\text{idx}} = 17$, $C_{\text{base}} = 13$, and $C_{\text{span}} = 29$ (all in characters).

For comparison, Table 9 reports the average input text length (n), span count (m), and average span length ($\bar{l}$) measured on reference annotations, aggregated over all sub-datasets within each dataset group.

B.5 Experiments with Qwen3 Reasoning

Table 11 summarizes our experiments with Qwen3-8B with reasoning disabled (our default model) vs. Qwen3-8B with reasoning enabled. We note that while in some cases the reasoning helped significantly (e.g. 47% improvement in soft-F1 score on NER with the INDEX method), the results occasionally got worse. Our manual inspection of results has shown that the reasoning trace sometimes got very long, exceeding the `max_token` limit, even after we increased it to 16,384 (e.g., when the model started to sequentially enumerate all the characters in the input sequence to get their indices), so it was not able to generate the final output.

B.6 GPT-5-mini Results

Table 12 presents the hard and soft F1 scores for the proprietary GPT-5-mini model. Note that LOGITMATCH and its variants cannot be applied to GPT-5-mini as it does not expose logits.

C Implementation Details

C.1 Prompts

The overview of the prompts we use for individual methods and tasks is provided in Table 13.

Method	soft	hard	gap	span	label	parse
MATCH	50.4	40.8	9.6	5.49	0.49	0.75
LOGITMATCH	50.2	40.5	9.7	**0.00**	**0.00**	2.96
MATCH-OCC	51.9	42.4	9.5	4.80	0.58	0.87
LOGITMATCH-OCC	51.5	41.9	9.6	**0.00**	**0.00**	2.94

Table 8: Soft/hard F1 (%), their gap (in points), and span/label/parse error rates (% of examples) of the *matching* methods, averaged over the four open models and all tasks (non-thinking, unstructured configuration).

Dataset	input chars (n)	spans (m)	span len ($\bar{l}$)
NER	112.0	1.9	8.8
GEC	867.1	14.1	4.2
ESA-MT	250.3	2.2	13.7
CPL	536.9	12.7	6.0

Table 9: Average input text length (n, characters), span count per example (m), and average span length ($\bar{l}$, characters) from reference annotations.

C.2 Methods

Below, we provide extra implementation details for individual methods.

Tagging For the *tagging* strategy, we parse XML-style tags using regular expressions. To map these predictions back to the original text, we estimate the span's location by stripping tags from the preceding output to calculate a running offset. Since models occasionally alter whitespace or hallucinate, we treat this calculated position as a heuristic rather than an absolute truth. If the text at the expected location does not match the tag content exactly, we search the full input for an *exact* occurrence of the identical span text and, if found, relocate the span to the occurrence closest to our initial estimate; otherwise, we drop the span. To keep the post-processing fair, we use **exact string search** only – no fuzzy matching is involved. A predicted span is therefore either (a) matched at its implied offset, (b) relocated to an exact occurrence of the identical text, or (c) dropped. This allows us to recover span boundaries even when the model's output deviates slightly from the input instead of having to discard the output completely.

LOGITMATCH The pseudocode for the LOGITMATCH algorithm (Section 4) is described in Algorithm 1. Note that for presentation purposes, the pseudocode does not account for tokenization issues described in Section 4.2. However, we account for these issues in our actual implementation in the project repository. We implement our method as a vLLM `LogitsProcessor`,[11] which allows us to combine the method with structured outputs.

C.3 Data Preprocessing

We made several adjustments to the MultiGEC dataset (Masciolini et al., 2025) that we used for the GEC task:

- The original MultiGEC dataset contains texts in 12 languages in total. We initially wanted to include all the languages in our experiments. However, we found that for the majority of languages, the inputs (text with errors) and the outputs (corrected texts) are not aligned. Only the *English* subset (Write & Improve corpus) contained the list of edits between the original and corrected sentences, which corresponds directly to the span labeling task. Therefore, we limited our dataset only to the English subset.
- Since we focus mainly on span identification, we ignore the corrected text for the replacement and missing edits. For the same reason, we also use only the high-level categories (replacement, missing, and unnecessary) and ignore the more fine-grained categories.
- The original MultiGEC dataset represents missing spans (label `M`) as zero-length spans, where the start and end index are equal and indicate the insertion point. Since matching-based methods cannot refer to an empty span, we instruct the model in the MATCH and MATCH-OCC prompts to output the token immediately preceding the insertion point instead. In post-processing, we advance the predicted start index past the output token to recover the insertion point. The evaluation metric (Section 5.4) treats each zero-length span as contributing a unit weight to the relevant F1 denominator, with a zero-length predicted span counting as a match for a zero-length gold span only when their positions coincide and (under hard matching) their labels agree. The tagging and indexing strategies do not receive this instruction and their outputs are parsed as-is.

[11] https://docs.vllm.ai/en/latest/design/logits_processors/

Method	NER				GEC				ESA-MT				CPL			
	8B	24B	70B	31B	8B	24B	70B	31B	8B	24B	70B	31B	8B	24B	70B	31B
TAG	**79.3**	79.4	**87.1**	92.6	**29.4**	**35.1**	**37.5**	**55.9**	21.3	22.2	24.7	**32.5**	40.2	53.1	79.5	93.3
INDEX	26.5	32.7	32.8	57.3	11.8	10.8	10.4	10.7	18.3	17.4	16.5	18.8	34.1	40.0	36.6	57.9
INDEX-ENRICHED	38.9	74.2	75.2	91.4	15.6	20.1	20.5	51.5	18.1	18.8	20.4	25.8	45.4	**70.1**	84.4	**98.6**
MATCH	79.2	**83.6**	85.4	90.7	15.6	21.4	22.7	42.2	24.3	24.5	25.7	29.7	42.8	43.8	48.5	51.1
MATCH-S	77.8	80.5	85.7	88.3	14.4	19.1	22.0	41.7	24.0	25.0	25.5	27.8	42.6	41.1	47.8	39.4
MATCH-OCC	78.4	83.0	85.4	91.6	19.2	24.0	23.6	45.4	23.8	25.4	25.1	30.0	**70.1**	69.1	**86.5**	94.7
MATCH-OCC-S	75.8	80.3	85.5	91.1	18.2	23.3	23.8	45.3	24.5	**25.6**	25.7	29.4	**70.1**	67.7	83.6	93.8
LOGITMATCH	78.7	80.6	83.4	92.5	18.7	24.7	21.2	44.4	23.9	24.8	25.7	30.7	40.1	42.9	47.4	51.1
LOGITMATCH-S	77.5	77.7	83.8	89.3	17.9	22.1	23.8	44.3	23.9	24.8	**25.8**	27.7	41.2	40.9	46.7	39.7
LOGITMATCH-OCC	78.1	79.7	82.9	**93.3**	22.9	26.0	22.8	47.8	23.7	25.3	24.5	30.6	68.6	68.9	86.3	94.7
LOGITMATCH-OCC-S	75.6	77.1	83.2	92.4	21.9	25.5	25.1	48.0	**24.8**	25.0	25.6	29.4	69.6	68.0	84.4	93.7

Table 10: Per-method **soft** F1 score in %. The best score for each model per dataset is bold.

Method	NER				GEC				ESA-MT				CPL			
	8B		8B-T		8B		8B-T		8B		8B-T		8B		8B-T	
	hard	soft	hard	soft	hard	soft	hard	soft	hard	soft	hard	soft	hard	soft	hard	soft
TAG	73.3	**79.3**	84.3	88.4	**27.2**	**29.4**	29.3	**41.7**	10.2	21.3	9.8	22.3	40.2	40.2	89.4	89.4
INDEX	19.1	26.5	69.8	74.1	10.4	11.8	6.7	8.0	7.5	18.3	7.6	17.7	34.1	34.1	67.6	67.6
INDEX-ENRICHED	33.7	38.9	78.5	84.3	11.0	15.6	22.9	25.1	7.5	18.1	9.9	22.7	45.4	45.4	**93.5**	**93.5**
MATCH	**74.1**	79.2	84.4	88.3	12.1	15.6	30.8	37.3	11.4	24.3	**10.8**	25.0	42.8	42.8	51.7	51.7
MATCH-S	72.8	77.8	**84.5**	88.4	12.0	14.4	34.6	37.4	10.7	24.0	10.4	**25.2**	42.6	42.6	51.8	51.8
MATCH-OCC	73.0	78.4	76.7	81.0	15.4	19.2	32.8	38.3	**11.9**	23.8	10.2	24.2	**70.1**	**70.1**	75.4	75.4
MATCH-OCC-S	70.8	75.8	77.1	81.4	14.9	18.2	**35.7**	38.6	11.1	24.5	10.7	24.1	**70.1**	**70.1**	75.8	75.8
LOGITMATCH	73.3	78.7	84.1	88.4	17.4	18.7	33.6	36.5	11.2	23.9	9.8	22.7	40.1	40.1	51.0	51.0
LOGITMATCH-S	71.9	77.5	84.4	**88.8**	16.8	17.9	34.8	37.8	10.8	23.9	9.9	22.8	41.2	41.2	51.1	51.1
LOGITMATCH-OCC	72.4	78.1	77.6	82.3	21.3	22.9	34.2	36.7	11.8	23.7	10.0	22.0	68.6	68.6	74.6	74.6
LOGITMATCH-OCC-S	70.1	75.6	77.7	82.3	20.4	21.9	35.2	37.9	11.2	**24.8**	9.8	22.0	69.6	69.6	74.8	74.8

Table 11: Qwen3-8B F1-score (hard, soft) in % with vs. without reasoning. Qwen3-8B (8B) has reasoning capabilities turned off, while Qwen3-8B-Think (8B-T) has them turned on. The best score for each model per dataset is bold.

Method	NER		GEC		ESA-MT		CPL	
	5-m		5-m		5-m		5-m	
	hard	soft	hard	soft	hard	soft	hard	soft
TAG	74.3	80.3	**37.4**	**41.3**	12.2	27.5	73.0	73.0
INDEX	38.3	43.3	7.7	9.2	8.0	19.0	48.8	48.8
INDEX-ENRICHED	78.9	**85.2**	27.1	32.6	11.3	24.9	83.2	83.2
MATCH	79.0	82.3	22.4	26.3	12.1	26.7	48.4	48.4
MATCH-S	77.2	80.9	19.8	23.4	12.3	27.2	43.5	43.5
MATCH-OCC	**79.5**	83.0	26.7	30.9	12.3	**27.8**	76.4	76.4
MATCH-OCC-S	76.1	79.8	25.6	29.5	**12.5**	27.6	**85.6**	**85.6**

Table 12: GPT-5-mini F1-score (hard, soft) in % per method and dataset. The best score per dataset is bold.

Algorithm 1 LogitMatch decoding algorithm

Require: Input $X = [x_1, x_2, \ldots, x_n]$, model $\mathcal{M}$

Initialize output sequence Y
while decoding is not finished **do**
    $t \leftarrow$ next token predicted by $\mathcal{M}(Y)$ ▷ **Decoding loop**
    Append t to Y
    if `'"text": "'` was decoded **then** ▷ **Select mode**
        $V_{\text{select}} \leftarrow \{x \mid x \in X\}$ ▷ Limit vocabulary to tokens present in input
        $x_i \leftarrow$ sample from $\mathcal{M}(Y)$ constrained to V_{select}
        Append x_i to Y
        $k \leftarrow 1$ ▷ Initialize relative offset
        loop ▷ **Copy mode**
            $V_{\text{copy}} \leftarrow \{x_{i+k}\} \cup \{\text{"}\}$ ▷ Allow next input token or closing quote
            $t_{\text{next}} \leftarrow$ sample from $\mathcal{M}(Y)$ constrained to V_{copy}
            Append t_{next} to Y
            if t_{next} is " **then**
                break ▷ Exit copy mode
            else
                $k \leftarrow k + 1$ ▷ Advance relative offset
            end if
        end loop
    end if
end while

Prompt template	
Structure	[Task description] Output Format: [Format] Labels: [Label list] Examples: 1. [Example input] -> [Example output] ... [Notes] [Input text]
Task definitions	
NER	Extract named entities (PERSON, ORG, LOC) from the text.
GEC	Identify grammatical errors in learner-written text.
ESA-MT	Identify translation errors by comparing the translation to the source text.
CPL	Find all text spans that match the given pattern queries.
Tagging strategies	
Format	<entity type="LABEL"></entity>
Note	IMPORTANT: Your output needs to include copy of the entire input text, including non-tagged parts. If you are not outputting any tags, you need to copy the input text literally. Surround the specific spans with the XML tags as required. Do not output any additional explanations or comments, start generating output straight away.
Example output	<entity type="PER">Lina Berg</entity> joined <entity type="ORG">AstraTech</entity> in <entity type="LOC">Stockholm</entity> after finishing her studies at <entity type="ORG">Northvale University</entity>.
Matching strategies	
Format	[{"text": "exact text span from input", "label": "category (PERSON, ORG, LOC)"}]
Note	Return a valid JSON array only.
Example output	[{"text": "Lina Berg", "label": "PER"}, {"text": "AstraTech", "label": "ORG"}, {"text": "Stockholm", "label": "LOC"}, {"text": "Northvale University", "label": "ORG"}]
For Match-Occ	
Format	[{"text": "exact span from input", "label": "category (PERSON, ORG, LOC)", "occurrence": "which occurrence (1, 2, 3...)"}]
Example output	[{"text": "Lina Berg", "label": "PER", "occurrence": 1}, {"text": "AstraTech", "label": "ORG", "occurrence": 1}, {"text": "Stockholm", "label": "LOC", "occurrence": 1}, {"text": "Northvale University", "label": "ORG", "occurrence": 1}]
Indexing strategies	
Format	[start:end] = LABEL
Note	IMPORTANT: Character positions are 0-indexed. - First character is at position 0 - Spaces count as characters - start is inclusive, end is exclusive
Example output	[0:9] = PER [17:26] = ORG [30:39] = LOC [71:91] = ORG

Table 13: Overview of prompts used for each strategy.